Objectifs généraux

Dans ce TP, nous allons apprendre en premier lieu à construire un modèle de régression linéaire en python puis un modèle de régression logistique.

Partie I : Régression Linéaire

Pour construire un modèle de régression linéaire en python, nous allons suivre cinq étapes :

- Lire et comprendre les données
- Visualisation des données
- Réalisation d'une régression linéaire simple
- Prédictions sur l'ensemble de test

Importation des bibliothèques à utiliser

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
```

I. Lecture et description des données

On considère le fichier de données suivant : « Company_data.csv »

1. Importation et affichage des données

```
advertising = pd.read_csv("Company_data.csv",sep = ',')
```

2. Affichage des 10 premiers ligne de la base

3. Dimension de la base

4. Information et description de la base

```
advertising.info()
advertising.describe()
```

II. Visualisation des données

1. Nous allons créer un Nuages de points (2 à 2) de toutes les colonnes et voir les colonnes qui sont les plus corrélées aux ventes.

```
sns.pairplot(advertising, x_vars=['TV', 'Radio','Newspaper'],
             y_vars='Sales', size=4, aspect=1, kind='scatter')
plt.show()
```

```
seaborn.pairplot(data,  x_vars=None, y_vars=None, kind='scatter',  height=2.5,
                        aspect=1,  size=None)
```

***data**: dataframe comportant les données où chaque colonne est une variable et chaque ligne est une observation.*

***{x, y}_vars** listes de noms de variables à utiliser séparément pour les lignes et les colonnes de la figure, c'est-à-dire pour réaliser un graphique non carré.*

***Kind :** type du graphique à créer Valeur possible : 'scatter', 'kde', 'hist', 'reg'*

2. On peut également extraire la relation entre la variable « sales » et les autres variables à l'aide d'une matrice nommé « heatmap »

```
sns.heatmap(advertising.corr(), annot = True)
plt.show()
```

3. **A partir du graphique et de cette matrice, quelle est la variable la plus corrélé avec la variable « sale »**

Exécutons le modèle de régression linéaire simple en utilisant la variable la plus corrélé comme variable caractéristique.

III. Réalisation d'une régression linéaire simple

Réalisation d'une régression linéaire simple

Équation de la régression linéaire simple

$$y = b_0 + b_1 X$$

Dans notre cas :

$$\ldots = b_0 + b_1 \ldots$$

Nous allons effectuer une régression linéaire simple en quatre étapes.

a) Créer X ( ) et Y ( ).
b) Créer la base d'apprentissage et la base de test.
c) Apprentissage du modèle
d) Evaluation.

1. Créer X et y

Tout d'abord, nous allons attribuer à notre variable caractéristique la valeur X et à notre variable cible la valeur y.

2. Créer la base d'apprentissage et la base de test.

Nous devons diviser notre base en deux ensembles (apprentissage et test). À l'aide de l'ensemble d'apprentissage, nous allons construire le modèle pour l'exécuter sur l'ensemble de test.

70% des données vont former la basse d'apprentissage et 30% de la base sera consacré pour la base de test.

Nous allons diviser les données en utilisant « `train_test_split` » de la bibliothèque « `sklearn.model_selection` ».

```
X_train_lm, X_test_lm, y_train_lm, y_test_lm =
train_test_split(X, y, train_size = 0.7, test_size = 0.3, random_state = 100)
```

```
train_test_split(*arrays, test_size=None, train_size=None, random_state=None,
                 shuffle=True, stratify=None)
```

test_size : float or int, default=None

- *Si « float », doit être compris entre 0,0 et 1,0 et il représente la proportion de l'ensemble de données à inclure dans la répartition du test.*
- *Si « int », représente le nombre absolu d'échantillons de test.*
- *Si « None », la valeur est définie comme le complément de la taille du train. Si « train_size » est également None, il sera fixé à 0,25.*

random_state : int, default=None *si vous ne spécifiez pas le random_state dans votre code, alors chaque fois que vous exécutez votre code, une nouvelle valeur aléatoire est générée et les données du train et du test auraient des valeurs différentes à chaque fois.*

Toutefois, si une valeur fixe est attribué comme random_state = 42, 100 , alors peu importe combien de fois vous exécutez votre code, le résultat serait le même .I. e, mêmes valeurs dans les ensembles de données train et test.

shuffle= bool, default=True : *Indique s'il faut ou non mélanger les données avant de les fractionner. Si shuffle=False, stratify doit être None.*

stratify= array-like, default=None : C'est-à-dire en respectant les proportions des deux classes dans les deux sous-ensembles.

Jetons un coup d'œil sur la taille de l'ensemble de données de formation,

```
X_train_lm.shape
```

Test

```
y_train_lm.shape
```

Pour une régression linéaire simple, nous devons ajouter une colonne pour effectuer l'ajustement de la régression correctement.

```
X_train_lm = X_train_lm.values.reshape(-1,1)
X_test_lm = X_test_lm.values.reshape(-1,1)
print(X_train_lm.shape)
print(X_test_lm.shape)
```

3. régression

```
lm = LinearRegression()
lm.fit(X_train_lm, y_train_lm)
```

1. Déterminer $b_0$et b_1

```
print("coeficient du modéle :",lm.intercept_)

# Slope value
print('pente :',lm.coef_)
```

Finalement, L'équation de la ligne droite que nous obtenons pour les valeurs ci-dessus est la suivante :

4. Evaluation du modèle

Pour l'évaluation du modèle, nous utilisons la valeur de R_caré (R2_score) :

Ce coefficient est compris entre 0 et 1, et croît avec l'adéquation de la régression au modèle:

– *Si le R^2 est proche de zéro, alors la droite de régression colle à 0% avec l'ensemble des points donnés.*

– *Si le R2 d'un modèle est de 0,50, alors la moitié de la variation observée dans le modèle calculé peut être expliquée par les points.*

– *Si le R^2 est de 1, alors la régression détermine 100% de la distribution des points.*

En pratique, il est impossible d'obtenir un R2 de 1 à partir de données empiriques. On considère qu'un R carré est élevé lorsqu'il se situe entre 0.85 et 1

```
y_train_pred = lm.predict(X_train_lm)
y_test_pred = lm.predict(X_test_lm)
print(r2_score(y_train_lm,y_train_pred))
print(r2_score(y_test_lm,y_test_pred))
```

Partie II : Régression Logistique

Objectif : sélection des variables pertinentes pour la prédiction de la présence d'une maladie du cœur.

Dans un nouveau notebook , importer les bibliothèques nécessaires.

1. Importation et chargement des données.

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix,
plot_confusion_matrix,accuracy_score
```

2. Dans cette partie, nous allons exploiter les données du fichier « demo_reglog.csv »

```
data = pd.read_csv("demo_reglog.csv",sep = ',')
```

3. **Afficher les cinq dernières lignes de la base**

4. **Description des données**

5. **Pourquoi la variable « cœur » ne figure pas dans la description des données**

6. Distribution des classes

```
print(data['coeur'].value_counts())
```

7. **Préparation des données : X contient les 3 variables âge, taux et angine**
   Y contient la variable cœur

8. Nous allons diviser les données en gardant 40% de la base pour le test et application de la régression logistique

```
modele_logit = LogisticRegression()
X_train,X_test,y_train,y_test = train_test_split(x,y,
test_size=0.4,random_state=100)
res=modele_logit.fit(X_train,y_train)
```

9. Affichage des paramètres de la régression logistique

```
print (res.intercept_)
print(res.coef_)
```

10. Affichage indexé des coefficients de chaque variable (âge, taux, angine)

```
print(pd.DataFrame({"var":x.columns,"coef":res.coef_[0]}))
```

11. Quelle variable influence le plus dans la détermination de l'existence ou non d'une anomalie de coeur.

12. Prédictions sur les données de test en utilisant notre modèle formé

```
preds = modele_logit.predict(X_test)
print(preds)
print (y_test)
```

13. Evaluation du modèle

a) Précision

```
modele_logit.score(X_test,y_test)
print(Précision: ',accuracy_score(y_test, preds))
```

b) Matrice de confusion

```
plot_confusion_matrix(modele_logit, X_test, y_test)
```